

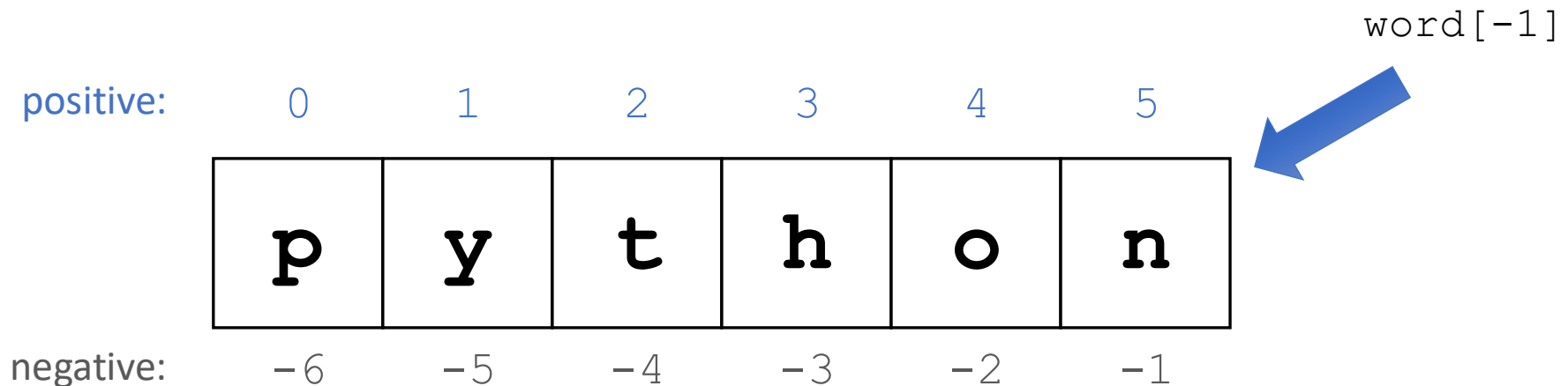
Slicing

INDEXING

- Each character in a string has a position, called its index
- Indexes start at 0, not 1
- Negative indexes count backward from the end: -1 is the last character

```
word = "python"  
print(word[0])  
print(word[-1])
```

p
n

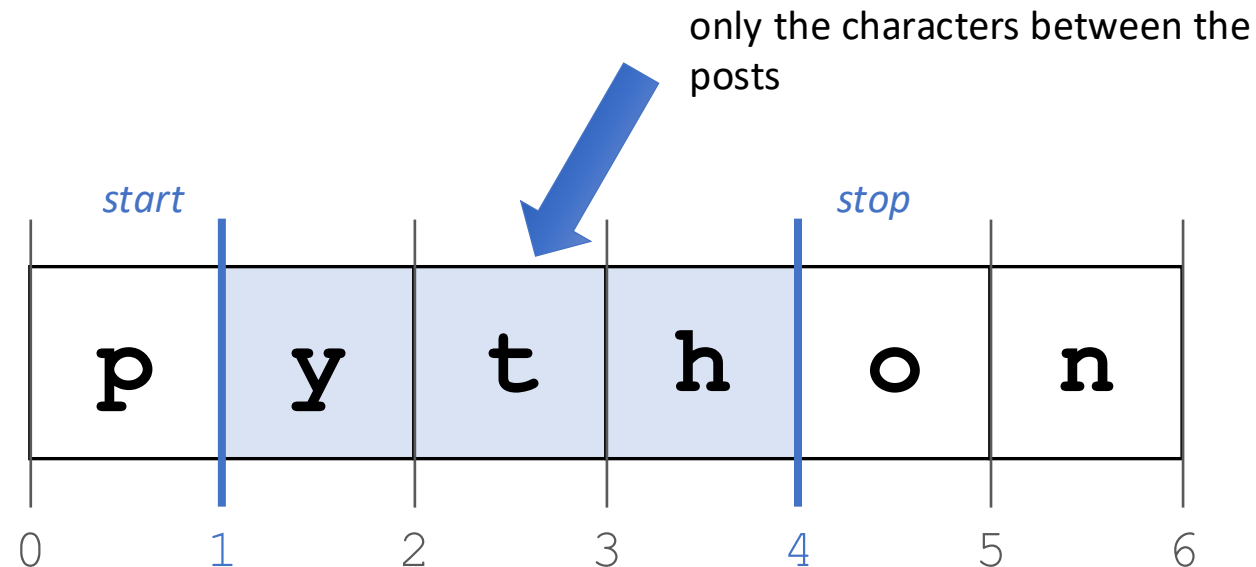


SLICES

- `word[start:stop]` gives the characters from `start` up to, but not including, `stop`
- Picture the indexes sitting between the characters, like fence posts
- The length of a slice is `stop - start`

```
word = "python"  
print(word[1:4])
```

yth



LEAVING THINGS OUT

- Leave out `start` and the slice begins at the start of the string
- Leave out `stop` and it runs to the end
- Leave out both and you get the whole string
- For any `n`, `word[:n] + word[n:]` is the whole string again

```
word = "python"  
print(word[:2])  
print(word[2:])  
print(word[:])  
print(word[:2] + word[2:])
```



the pieces fit back
together

```
py  
thon  
python  
python
```

NEGATIVE INDEXES IN SLICES

- Negative indexes work in slices too
- `word[-3:]` means “the last three characters”
- `word[:-2]` means “everything except the last two”
- Useful when you care about the end of a string but don't know its length



```
word = "python"  
print(word[-3:])  
print(word[:-2])  
  
file = "notes.txt"  
print(file[-4:])
```

```
hon  
pyth  
.txt
```

STEPPING

- A slice can take a third number:
`word[start:stop:step]`
- `step` says how far to move each time (the default is 1)
- A negative step walks backward, so
`word[::-1]` is the string reversed

```
word = "python"  
print(word[::2])  
print(word[1::2])  
print(word[::-1])
```

```
pto  
yhn  
nohtyp
```



every character, back to front

SLICES ARE FORGIVING

- Indexing past the end of a string is an error
- Slicing past the end is not: Python just stops at the end of the string
- A slice that selects nothing gives the empty string, ''
- Handy when a string might be shorter than you expect



```
word = "python"  
print(word[10])
```

```
IndexError: string index  
out of range
```



```
print(word[2:10])  
print(word[10:])  
print(len(word[10:]))
```

```
thon
```

```
0
```

STRINGS CAN'T BE CHANGED

- Strings are immutable: you can't change a character in place
- Instead, build a new string from slices and concatenation
- String methods like `upper()` also return a new string; the original is unchanged

```
word = "python"  
word[0] = "J"
```

`TypeError: 'str' object does not support item assignment`



a new string

```
new_word = "J" + word[1:]  
print(new_word)  
print(word)
```

Jython
python

PREDICT THE OUTPUT

- Work these out on paper before running them
- Then check your answers in Colab
- Write out the indexes of "slicing" first; it helps!



two of these are trick questions

```
s = "slicing"
print(s[1:4])
print(s[-3:])
print(s[::3])
print(s[:2] + s[-2:])
print(s[4:100])
print(s[5:2])
```

YOUR TURN

in-class exercises: return values, conditionals, and strings